

Automate Login Test Case with Valid Credentials (Static Data)

This test case represents a basic login scenario using static credentials.

Steps: Identify the pages involved and create their corresponding **Page Object Classes**.

1. *HomePage:*

- Update this Page Object Class by Adding the Login link element and a method to click on it.

2. *LoginPage:*

- Create a new [LoginPage](#) Page Object Class that contains the web elements and methods required to enter the email, enter the password, and click the login button.

3. *MyAccountPage:*

- If the login is successful, create a new [MyAccountPage](#) Page Object Class to validate that the “My Account” header is displayed.
- The method `isMyAccountPageExists()` should not perform direct validation. It should simply return a [boolean](#) value or throw an exception if the captured heading text “My Account” is not found.

```
public boolean isMyAccountPageExists() {  
    try  
    {  
        return(msgHeading.isDisplayed());  
    }  
    catch (Exception e) {  
        return(false);  
    }  
}
```

- The actual validation (assertion) logic should be implemented in the test class (**test layer**), not in the Page Object, to ensure a clear separation between **test logic** and **page behavior**.

Implement a data-driven login test that covers all positive and negative cases using valid and invalid data inputs

ExcelUtility Class – Utilities Package

The [ExcelUtility](#) class is implemented under the [utilities](#) package to provide reusable methods for Excel file operations such as:

- **Finding the number of rows** in a sheet.
- **Finding the number of cells** in a row.
- **Reading and updating data** in Excel sheets.

Usage: All methods in this class are **instance methods** (non-static). To access them, an object of [ExcelUtility](#) must be created.

Excel Sheet Path Handling

1. The **Excel file path** is captured through the class constructor.
2. The constructor assigns the provided path to an internal variable for use across all methods.

```
public ExcelUtility(String path) {  
    this.path = path;  
}
```

Additional Functionalities

While updating data in a cell, the utility checks:

- Existence of the **Excel file**
- Existence of the **sheet**
- Existence of the **row**

```
public void setCellData(String sheetName, int rownum, int colnum, String data) throws IOException {  
    File xlfile = new File(path);  
    //If Excel file not exists then create new Excel file  
    if(!xlfile.exists())  
    {  
        workbook = new XSSFWorkbook();  
        fo = new FileOutputStream(path);  
        workbook.write(fo);  
    }  
  
    fi = new FileInputStream(path);  
    workbook = new XSSFWorkbook();  
  
    //If Sheet not exists then create new sheet  
    if(workbook.getSheetIndex(sheetName)==-1)  
        workbook.createSheet(sheetName);  
    workbook.getSheet(sheetName);  
  
    //If Row not exists then create new row  
    if(sheet.getRow(rownum)==null)  
        sheet.createRow(rownum);  
    sheet.getRow(rownum);  
}
```

MyAccountPage – Page Object Class Updates

1. Add the logout button/link WebElement.
2. Create a method to perform the **click action** on the logout WebElement.

DataProviders Class – Explanation

- The **DataProviders** class contains only the **@DataProvider** methods used to supply test data to TestNG test cases.
- Each **@DataProvider** must have a **unique name**, for example: **@DataProvider(name = "LoginData")**
- The Excel file path is stored in a **variable**.
- Create an object of the **ExcelUtility** class to read data from the Excel file.

- During the creation of the object, the Excel file path is passed to the [ExcelUtility](#) **constructor**, which initializes the file and prepares it for read/write operations.
- Retrieve the data from the Excel sheet and store it in a **two-dimensional Object array**, which is then returned by the [DataProvider](#) method.
- Initially, data was **hardcoded** inside the two-dimensional array, but now it is **dynamically** read from Excel using the [ExcelUtility](#) class.
- The `dataProviderClass = DataProviders.class` attribute is used to access a [DataProvider](#) method located in a different class. We include this parameter because the [DataProvider](#) method is defined in the [DataProviders](#) class, not in the `TC_3_LoginDDT` test class.

```
@DataProvider(name="LoginData")
public String [][] getData() throws IOException {
    String path = ".\\testData\\Opencart_LoginData.xlsx"; //Taking Excel File from testData folder
    ExcelUtility xlutil = new ExcelUtility(path); //Creating an object from ExcelUtility class

    int totalrows = xlutil.getRowCount("Sheet1");
    int totalcols = xlutil.getCellCount("Sheet1", 1);

    String logindata[][] = new String[totalrows][totalcols];
    //Created two dimension array which can store the data user ad password

    for(int i=1; i<=totalrows; i++)
    {
        for(int j=0; j<totalcols; j++)
        {
            logindata[i-1][j] = xlutil.getCellData("Sheet1", i, j);
        }
    }
    return logindata; //Returning two dimension array
}
```

Validation Logic in the “TC_3_LoginDDT” Test Class

For each data set (valid or invalid), the test must validate two possible outcomes: **Valid** data and **Invalid** data

- Valid data + Successful login → **Pass** (positive scenario) → **Logout required**
- Valid data + Unsuccessful login → **Fail** (positive scenario – unexpected behavior)
- Invalid data + Successful login → **Fail** (negative scenario – unexpected behavior) → **Logout required**
- Invalid data + Unsuccessful login → **Pass** (negative scenario – expected behavior)

```
if(expect.equalsIgnoreCase("Valid"))
{
    if(targetPage == true)
    {
        maac.clickLogout();
        Assert.assertTrue(true);
    }
    else
        Assert.assertTrue(false);
}
```

```
if(expect.equalsIgnoreCase("Invalid"))
{
    if(targetPage == true)
    {
        maac.clickLogout();
        Assert.assertTrue(false);
    }
    else
        Assert.assertTrue(true);
}
```